SIMATS ENGINEERING

ASSESSMENT TOOL 1:

Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3)
Question Count: 4

Total Marks: 40

Code of Conduct

I am Thirukumaran S (192521149)(Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Thirukumaran S

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Cl

Experiment 1: Hospital Information System (SaaS)

Aim

To create a simple cloud-hosted Hospital Information System (HIS) for SIMATS Hospital that demonstrates the Software-as-a-Service (SaaS) model, capturing patient name, age, address, phone, email, attender name, attender phone, doctor name, disease and other details.

Requirements

- Cloud Service Provider account (Google Firebase Hosting – free Spark plan; AWS Amplify / S3 or Microsoft Azure Static Web Apps can be used equally)
- Web browser (Chrome / Edge) and a text editor (VS Code)
- Node.js and Firebase CLI (npm install -g firebase-tools)
- Files: hospital.html and style.css

Theory

Software as a Service (SaaS) is a cloud delivery model in which the provider hosts the application, manages servers, storage, security and updates, and users access the software through a web browser without installing anything locally. Examples include Gmail, Salesforce and Google Workspace. In a hospital, a SaaS-based HIS allows registration desks, doctors and nurses in different locations to use one centrally hosted system, pay on a subscription basis, and receive updates automatically.

In this experiment the application front end is developed with HTML, CSS and JavaScript and is published on a cloud hosting service. The browser Web Storage API stores records for demonstration; in production the same form would post to a cloud database such as Firestore, Amazon RDS or Azure SQL.

Procedure

1. Sign in to the Firebase console (console.firebase.google.com) and click Add project; name it simats-his.
2. Create a folder simats-his on the local machine and save hospital.html (renamed index.html) and style.css in a sub-folder named public.
3. Install the CLI: npm install -g firebase-tools, then run firebase login.
4. Inside the folder run firebase init hosting, select the simats-his project, enter public as the public directory and answer No to single-page-app rewrite.
5. Deploy using firebase deploy. The CLI prints a hosting URL such as https://simats-his.web.app.
6. Open the URL in a browser, register the patients, search the records and test deletion.

7.Capture screenshots of the form, the filled form and the record list.

Program – hospital.html

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>SIMATS Hospital Information System</title>
<meta name="viewport" content="width=device-width, initial-scale=1"><link rel="stylesheet"
href="style.css"></head>
<body>
<header><h1> SIMATS Hospital – Hospital Information System (SaaS)</h1><span>Hosted on Cloud |
Role: Front Desk</span></header>
<main>
<div class="card"><h2>Patient Registration</h2>
<div class="grid">
<div><label>Patient Name *</label><input id="pname"></div>
<div><label>Age *</label><input id="age" type="number" min="0" max="120"></div>
<div><label>Gender</label><select
id="gender"><option>Male</option><option>Female</option><option>Other</option></select></div>
<div><label>Phone *</label><input id="phone"></div>
<div><label>Email</label><input id="email" type="email"></div>
<div><label>Admission Date</label><input id="adate" type="date"></div>
<div style="grid-column:span 3"><label>Address</label><input id="address"></div>
<div><label>Attender Name</label><input id="aname"></div>
<div><label>Attender Phone</label><input id="aphone"></div>
<div><label>Department</label><select id="dept"><option>General
Medicine</option><option>Cardiology</option><option>Orthopaedics</option><option>Paediatrics</opti
on><option>Neurology</option></select></div>
<div><label>Doctor Name</label><select id="doctor"><option>Dr. R. Karthik</option><option>Dr. S.
Meena</option><option>Dr. A. Prasad</option><option>Dr. L. Nandhini</option></select></div>
<div><label>Disease / Diagnosis</label><input id="disease"></div>
<div><label>Ward / Bed</label><input id="ward" placeholder="e.g. B-204"></div>
</div>
<button onclick="addPatient()">Register Patient</button><button class="sec"
onclick="clearForm()">Clear</button>
</div>
<div class="card"><h2>Patient Records (<span id="count">0</span>)</h2>
<input id="search" placeholder="Search by name, doctor or disease..." oninput="render()"
style="margin-bottom:10px">
<table><thead><tr><th>ID</th><th>Patient</th><th>Age/Sex</th><th>Phone</th><th>Email</th><th>Atten
der</th><th>Doctor</th><th>Disease</th><th>Ward</th><th></th></tr></thead><tbody
id="rows"></tbody></table>
</div></main>
<script>
var KEY="simats_patients";
function load(){try{return JSON.parse(localStorage.getItem(KEY))||[]}catch(e){return[]}}
function save(d){localStorage.setItem(KEY,JSON.stringify(d))}
function v(id){return document.getElementById(id).value.trim()}
function addPatient(){
 if(!v("pname")||!v("age")||!v("phone")){alert("Patient name, age and phone are
required");return}
  if(!/^[0-9]{10}$/.test(v("phone"))){alert("Phone must be 10 digits");return}
  var d=load();

d.push({id:"P"+String(d.length+1001),name:v("pname"),age:v("age"),gender:v("gender"),phone:v("phon
e"),email:v("email"),

 address:v("address"),aname:v("aname"),aphone:v("aphone"),dept:v("dept"),doctor:v("doctor"),disease
:v("disease"),ward:v("ward"),date:v("adate")});
 save(d);clearForm();render();
}
function
clearForm(){["pname","age","phone","email","address","aname","aphone","disease","ward","adate"].fo
rEach(function(i){document.getElementById(i).value=""})}
function del(i){var d=load();d.splice(i,1);save(d);render()}
```

Cl

```
function render(){
  var d=load(),q=v("search").toLowerCase(),h="";
  d.forEach(function(p,i){
    if(q&&(p.name+p.doctor+p.disease).toLowerCase().indexOf(q)<0)return;
    h+="<tr><td><span
class='badge'>"+p.id+"</span></td><td>"+p.name+"</td><td>"+p.age+"/"+p.gender.charAt(0)+"</td><td>
"+p.phone+"</td><td>"+p.email+
    "</td><td>"+p.aname+"
("+p.aphone+")</td><td>"+p.doctor+"</td><td>"+p.disease+"</td><td>"+p.ward+"</td><td><button
class='del' onclick='del("+i+")'>Delete</button></td></tr>";
  });

document.getElementById("rows").innerHTML=h;document.getElementById("count").textContent=d.length;
}
render();
</script></body></html>
```

Program – style.css (shared by both applications)

```
*{box-sizing:border-box}body{margin:0;font-family:Segoe UI,Arial,sans-
serif;background:#eef2f7;color:#1f2937}
header{background:#0b5394;color:#fff;padding:14px 28px;display:flex;justify-content:space-
between;align-items:center}
header h1{margin:0;font-size:20px}header span{font-size:13px;opacity:.85}
main{max-width:1100px;margin:18px auto;padding:0 16px}
.card{background:#fff;border-radius:8px;padding:18px;margin-bottom:18px;box-shadow:0 1px 4px
rgba(0,0,0,.12)}
.card h2{margin:0 0 12px;font-size:16px;color:#0b5394;border-bottom:2px solid #e5e7eb;padding-
bottom:6px}
.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:12px}
label{font-size:12px;font-weight:600;display:block;margin-bottom:3px}
input,select,textarea{width:100%;padding:7px;border:1px solid #cbd5e1;border-radius:4px;font-
size:13px}
button{background:#0b5394;color:#fff;border:0;padding:8px 18px;border-
radius:4px;cursor:pointer;font-size:13px;margin-top:12px;margin-right:6px}
button.sec{background:#6b7280}button.del{background:#c0392b;padding:3px 8px;margin:0;font-
size:11px}
table{width:100%;border-collapse:collapse;font-size:12px}th{background:#0b5394;color:#fff;text-
align:left;padding:6px}
td{padding:6px;border-bottom:1px solid #e5e7eb}.total{text-align:right;font-weight:700;font-
size:15px;margin-top:8px}
.badge{background:#dbeafe;color:#1e40af;padding:2px 7px;border-radius:10px;font-size:11px}
```

Deployment Commands

```
npm install -g firebase-tools
firebase login
mkdir simats-his && cd simats-his
firebase init hosting   # public directory: public
cp hospital.html public/index.html && cp style.css public/
firebase deploy
```

Cl

Output

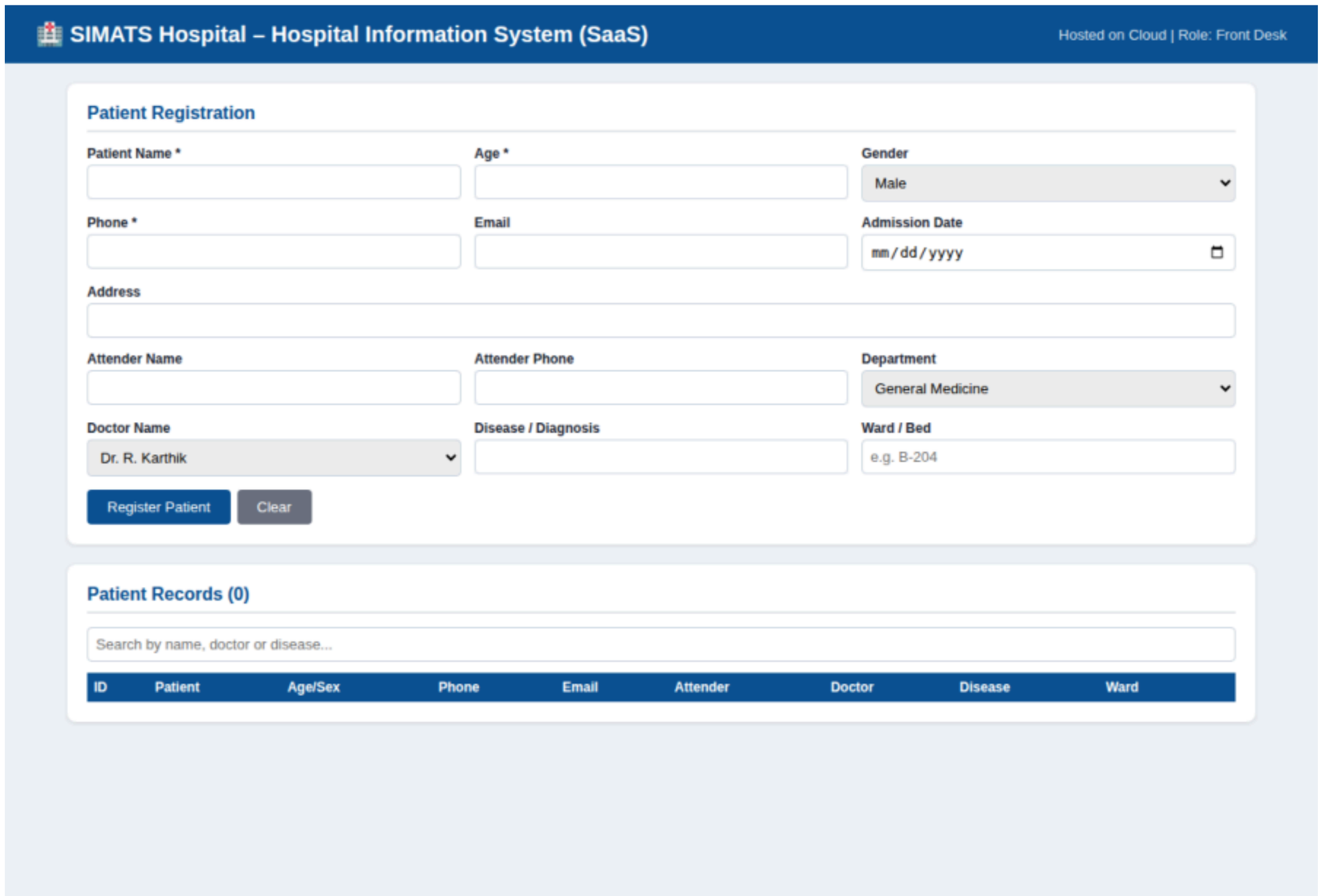


Figure 1.1 – Patient registration form of the SIMATS Hospital Information System

Cl

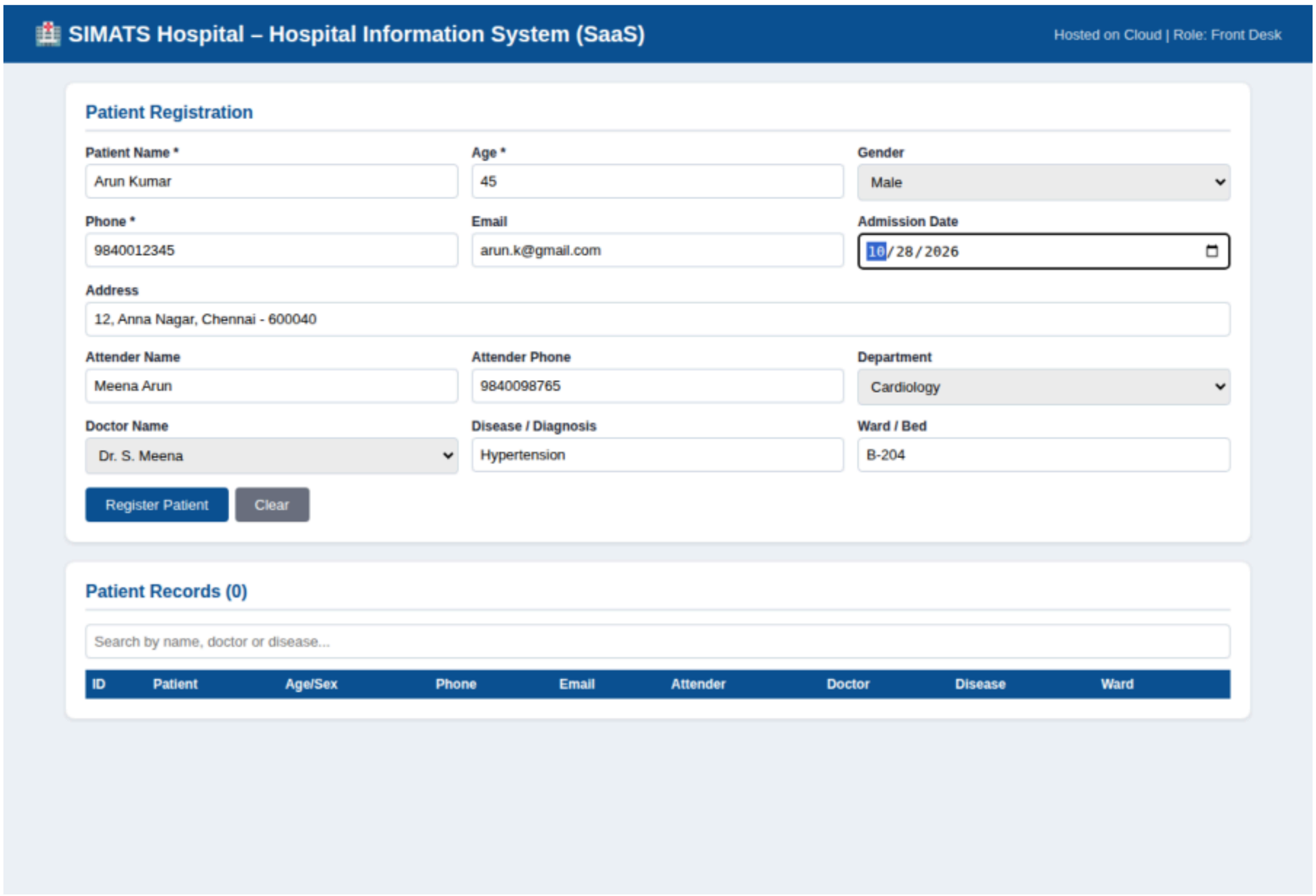


Figure 1.2 – Form filled with patient, attender and doctor details

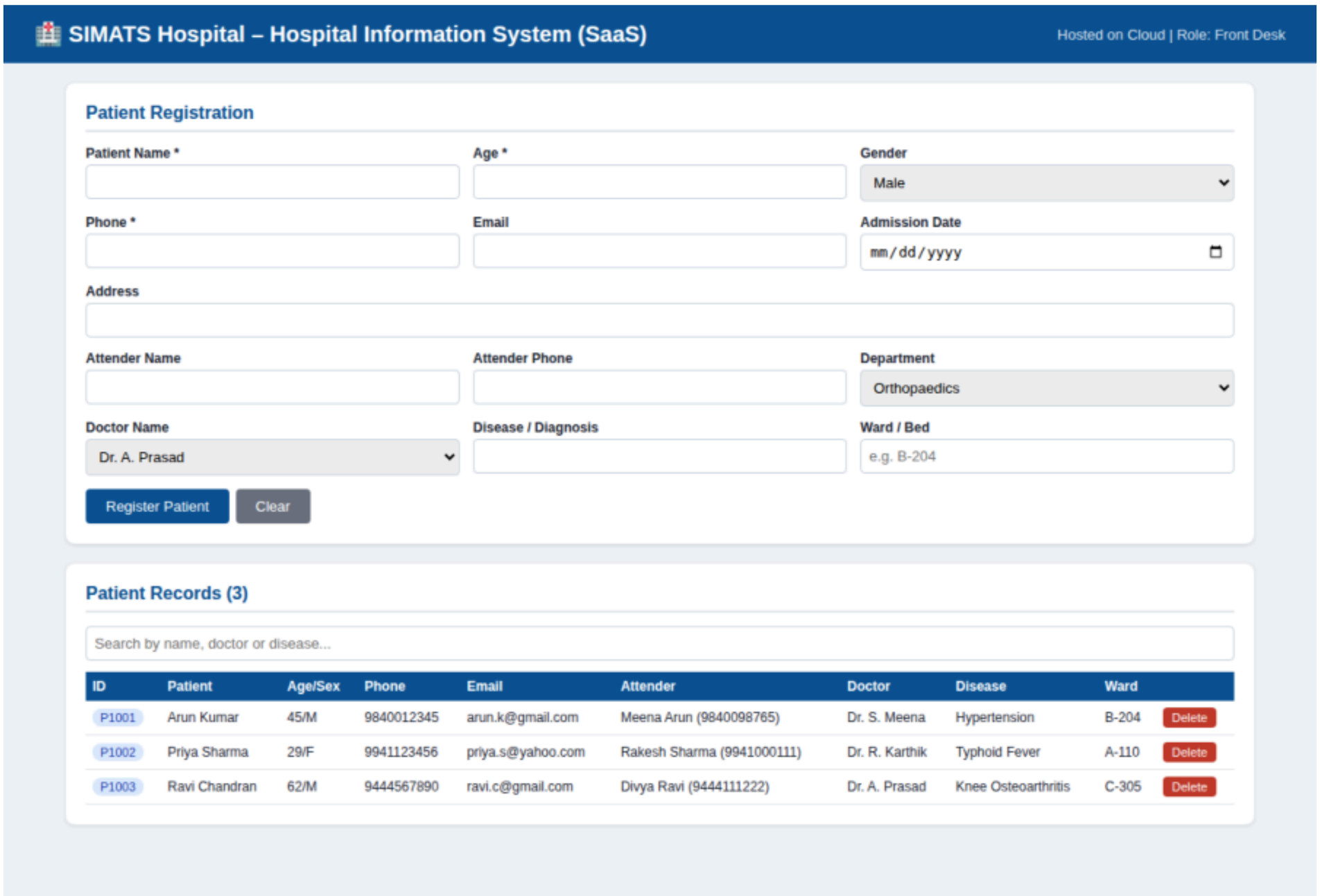


Figure 1.3 – Patient records table after registering three patients

Cl

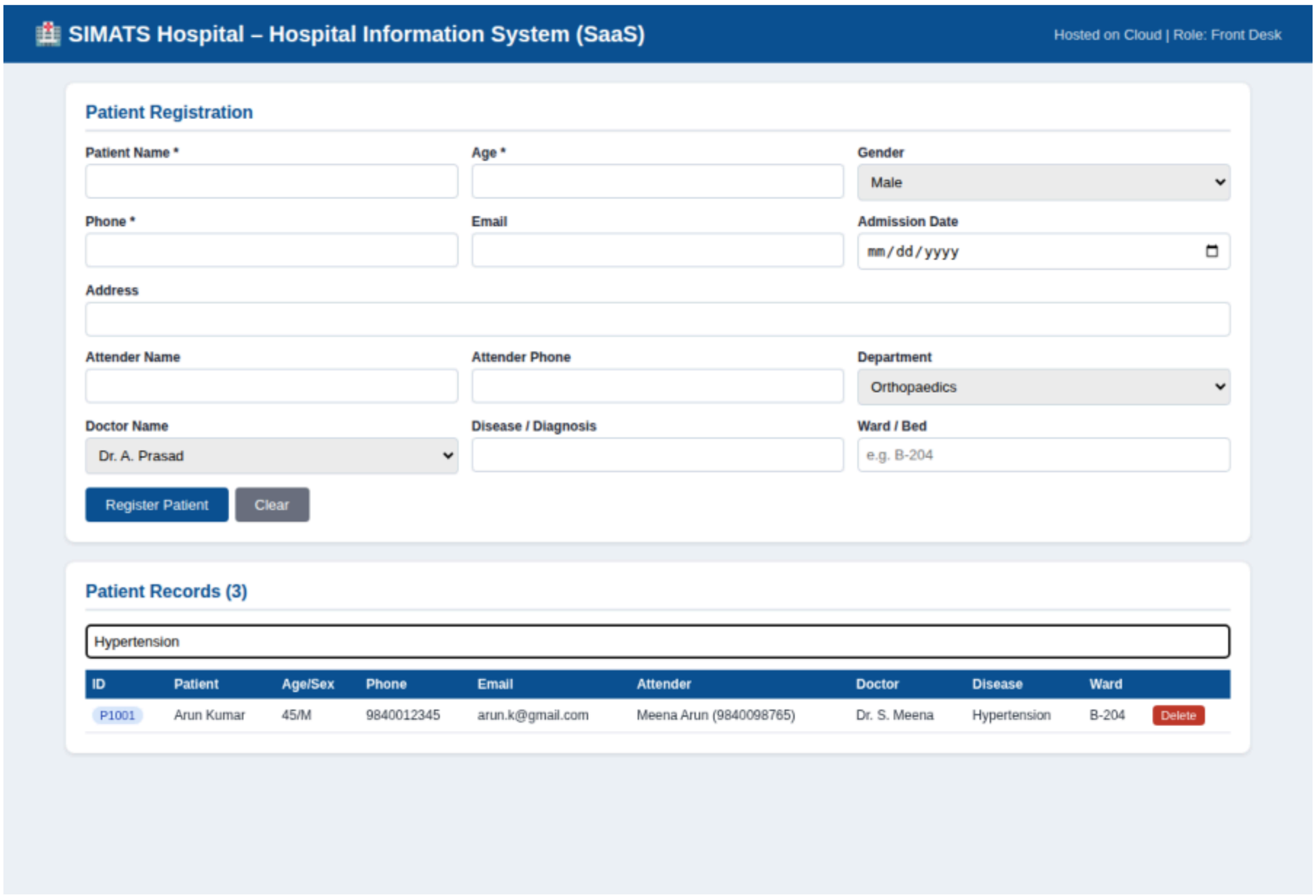


Figure 1.4 – Search result filtered by disease

Result

A SaaS-based Hospital Information System for SIMATS Hospital was developed and published on a cloud hosting platform. It can be accessed from any browser and successfully stores patient, attender and doctor details.

Experiment 2: Online Super Market (SaaS)

Aim

To create a simple cloud-hosted online supermarket application that demonstrates SaaS, with a customer template (name, address, phone, email) and an item classification form (category, item, quantity, price).

Requirements

- Cloud Service Provider account (Firebase Hosting / AWS Amplify / Azure Static Web Apps)
- Web browser and text editor
- Files: market.html and style.css

Theory

An online supermarket is a typical e-commerce SaaS workload. The provider hosts the catalogue, shopping cart and order management in the cloud and customers use them through a browser. Benefits of the cloud model include elastic scaling during festival-season traffic, pay-as-you-go pricing and no on-premise server maintenance. Items are classified into categories (Fruits & Vegetables, Dairy & Eggs, Grocery & Staples, Beverages, Snacks, Household) so that the item list and unit price change dynamically with the selected category.

Procedure

1. Create a new Firebase project named freshmart-online (or an S3 bucket / Azure Static Web App).
2. Save market.html as index.html and style.css in the public folder.
3. Run firebase init hosting and then firebase deploy.
4. Open the generated URL, enter the customer details and choose a category; the item list and price update automatically.
5. Add several items to the cart, verify the grand total and click Place Order.
6. Check that the order appears in Order History and capture the screenshots.

Program – market.html

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><title>FreshMart Online Supermarket</title>
<meta name="viewport" content="width=device-width, initial-scale=1"><link rel="stylesheet"
href="style.css"></head>
<body>
<header style="background:#1e8449"><h1> FreshMart – Online Super Market (SaaS)</h1><span>Hosted
on Cloud | Customer Portal</span></header>
<main>
<div class="card"><h2>Customer Details</h2><div class="grid">
<div><label>Customer Name *</label><input id="cname"></div>
<div><label>Phone *</label><input id="cphone"></div>
```

```
<div><label>Email</label><input id="cemail" type="email"></div>
<div style="grid-column:span 3"><label>Delivery Address *</label><input
id="caddr"></div></div></div>
<div class="card"><h2>Item Classification Form</h2><div class="grid">
<div><label>Category</label><select id="cat" onchange="fillItems()"></select></div>
<div><label>Item</label><select id="item" onchange="showPrice()"></select></div>
<div><label>Unit Price (₹)</label><input id="price" readonly></div>
<div><label>Quantity</label><input id="qty" type="number" min="1" value="1"></div></div>
<button onclick="addToCart()" style="background:#1e8449">Add to Cart</button></div>
<div class="card"><h2>Shopping Cart</h2>
<table><thead><tr><th>Category</th><th>Item</th><th>Qty</th><th>Price</th><th>Amount
(₹)</th><th></th></tr></thead><tbody id="cart"></tbody></table>
<div class="total">Grand Total: ₹<span id="total">0.00</span></div>
<button onclick="placeOrder()" style="background:#1e8449">Place Order</button></div>
<div class="card"><h2>Order History</h2>
<table><thead><tr><th>Order
ID</th><th>Customer</th><th>Phone</th><th>Address</th><th>Items</th><th>Total
(₹)</th><th>Status</th></tr></thead><tbody id="orders"></tbody></table></div>
</main>
<script>
var CATALOG={
 "Fruits & Vegetables":{"Apple (1 kg)":180,"Banana (1 dozen)":60,"Tomato (1 kg)":40,"Onion (1
kg)":35},
 "Dairy & Eggs":{"Milk (1 L)":56,"Curd (500 g)":35,"Butter (100 g)":58,"Eggs (12)":84},
 "Grocery & Staples":{"Basmati Rice (5 kg)":520,"Wheat Flour (5 kg)":260,"Toor Dal (1
kg)":150,"Sunflower Oil (1 L)":145},
 "Beverages":{"Tea Powder (250 g)":120,"Coffee (200 g)":280,"Orange Juice (1 L)":110},
 "Snacks":{"Potato Chips":20,"Biscuits Pack":30,"Chocolate Bar":50},
 "Household":{"Detergent (1 kg)":130,"Dish Wash Gel":95,"Toilet Cleaner":99}};
var cart=[];
function $(i){return document.getElementById(i)}
function init(){var
c=$("cat");Object.keys(CATALOG).forEach(function(k){c.innerHTML+="<option>"+k+"</option>"});fillIt
ems();renderOrders()}
function fillItems(){var
it=$("item");it.innerHTML="";Object.keys(CATALOG[$("cat").value]).forEach(function(k){it.innerHTML
+="<option>"+k+"</option>"});showPrice()}
function showPrice(){$("price").value=CATALOG[$("cat").value][$("item").value]}
function addToCart(){var q=parseInt($("qty").value);if(!q||q<1){alert("Enter valid
quantity");return}

cart.push({cat:$("cat").value,item:$("item").value,qty:q,price:parseFloat($("price").value)});rend
erCart()}
function renderCart(){var h="",t=0;cart.forEach(function(c,i){var a=c.qty*c.price;t+=a;

h+="<tr><td>"+c.cat+"</td><td>"+c.item+"</td><td>"+c.qty+"</td><td>"+c.price.toFixed(2)+"</td><td>
"+a.toFixed(2)+"</td><td><button class='del'
onclick='cart.splice("+i+",1);renderCart()'>Remove</button></td></tr>"});
 $("cart").innerHTML=h;$("total").textContent=t.toFixed(2)}
function orders(){try{return JSON.parse(localStorage.getItem("fm_orders"))||[]}catch(e){return[]}}
function placeOrder(){if(!$("cname").value||!$("cphone").value||!$("caddr").value){alert("Fill
customer name, phone and address");return}
 if(!cart.length){alert("Cart is empty");return}
 var o=orders(),t=0;cart.forEach(function(c){t+=c.qty*c.price});

 o.push({id:"ORD"+(5001+o.length),name:$("cname").value,phone:$("cphone").value,addr:$("caddr").val
ue,items:cart.map(function(c){return c.item+" x"+c.qty}).join(", "),total:t.toFixed(2)});
 localStorage.setItem("fm_orders",JSON.stringify(o));cart=[];renderCart();renderOrders()}
function renderOrders(){var h="";orders().forEach(function(o){h+="<tr><td><span
class='badge'>"+o.id+"</span></td><td>"+o.name+"</td><td>"+o.phone+"</td><td>"+o.addr+"</td><td>"+
o.items+"</td><td>"+o.total+"</td><td>Confirmed</td></tr>"});$("orders").innerHTML=h}
init();
</script></body></html>
```

Cl

Output

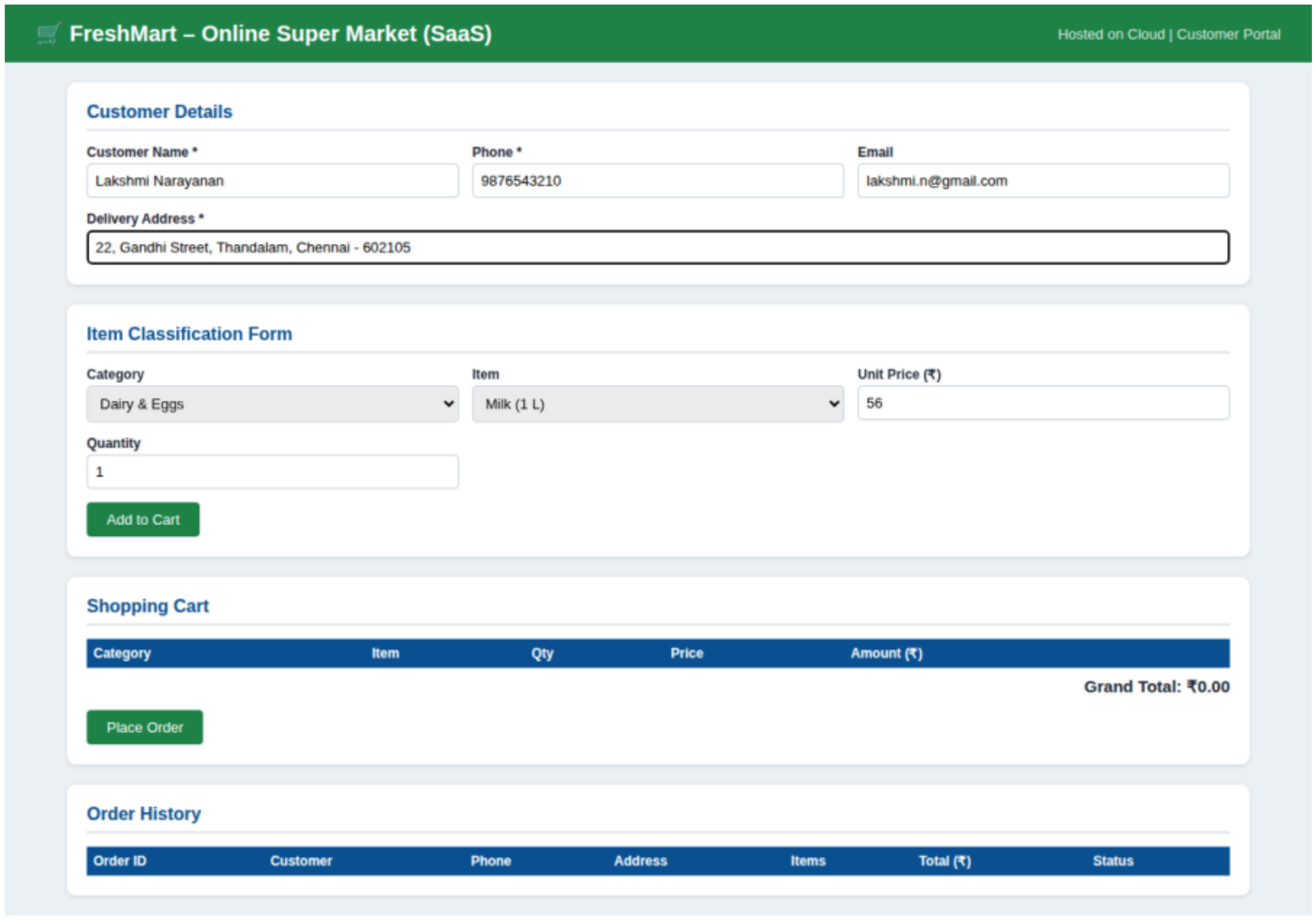


Figure 2.1 – Customer template and item classification form

Cl

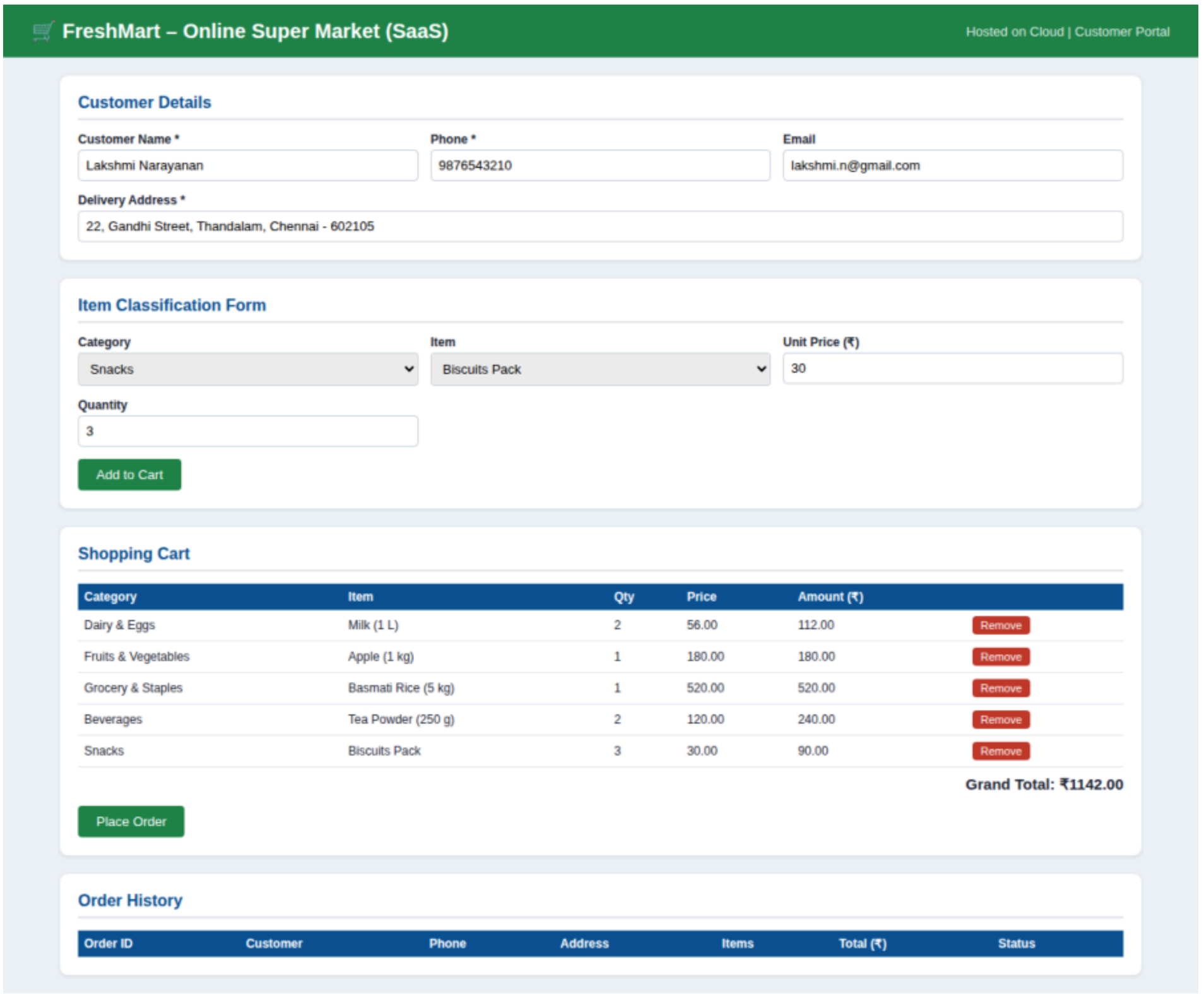


Figure 2.2 – Shopping cart with five items and grand total

Cl

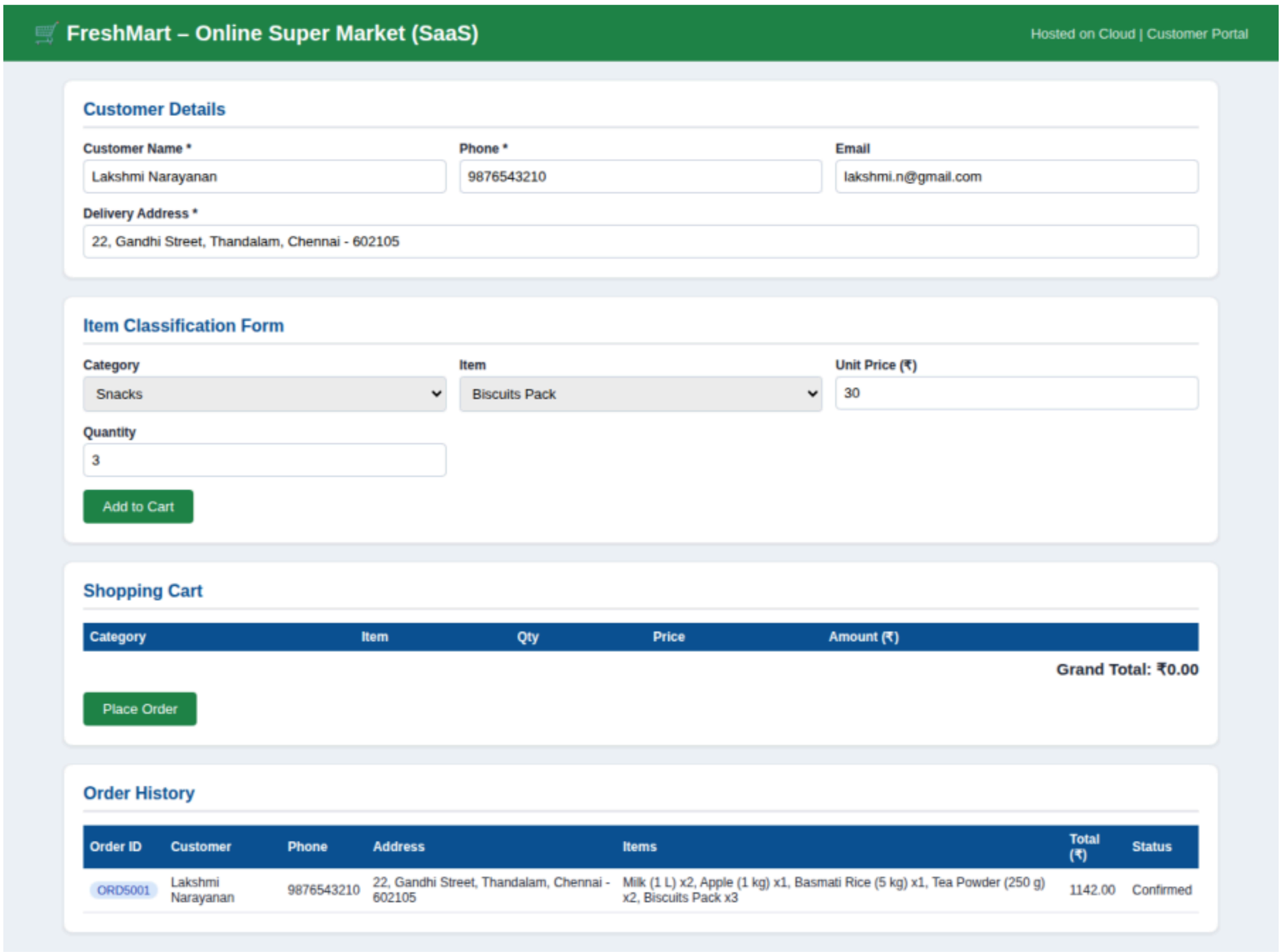


Figure 2.3 – Order confirmed and listed in Order History (total ₹1142.00)

Result

The online supermarket SaaS application with customer details and item classification was developed, deployed on the cloud and tested successfully.

Cl

Experiment 3: Virtualization – Create a VM (3 CPU, 10 GB RAM, 10 GB Disk)

Aim

To install a Type-2 hypervisor (Oracle VirtualBox) on a host operating system and create a virtual machine with 3 CPU cores, 10 GB RAM and a 10 GB virtual disk.

Requirements

- Host machine (Windows 10/11 or Linux) with hardware virtualization (Intel VT-x / AMD-V) enabled in BIOS
- At least 16 GB physical RAM and 4+ CPU cores, so that the host stays responsive
- Oracle VirtualBox 7.x (Type-2 hypervisor) – virtualbox.org
- Guest OS ISO – Ubuntu Server 22.04 LTS (a 10 GB disk is enough for the server edition; the desktop edition needs about 25 GB)

Theory

Virtualization abstracts physical hardware so that several isolated virtual machines can run on one machine. A hypervisor creates and manages the VMs. A Type-1 (bare-metal) hypervisor such as VMware ESXi or Microsoft Hyper-V runs directly on hardware, whereas a Type-2 (hosted) hypervisor such as VirtualBox or VMware Workstation runs as an application on top of a host operating system.

Feature	Type-1 Hypervisor	Type-2 Hypervisor
Runs on	Bare hardware	Host operating system
Performance	Higher	Slightly lower
Use case	Data centres, cloud	Desktops, labs, testing
Examples	ESXi, Xen, KVM, Hyper-V	VirtualBox, VMware Workstation

Procedure (GUI)

1. Enable virtualization (VT-x / AMD-V) in the BIOS/UEFI setup and boot the host OS.
2. Download and install Oracle VirtualBox together with the Extension Pack; accept the default options.
3. Open VirtualBox and click New. Enter Name: Ubuntu-VM, select the Ubuntu 22.04 ISO, Type: Linux, Version: Ubuntu (64-bit).
4. On Hardware, set Base Memory to 10240 MB and Processors to 3.
5. On Hard Disk, choose Create a Virtual Hard Disk Now, size 10 GB, type VDI, and click Finish.

6. Select the VM, click Settings > System > Processor and confirm 3 CPUs; Settings > Storage to confirm the ISO is attached.
7. Click Start, install Ubuntu and reboot into the installed system.
8. Verify the resources inside the guest using nproc, free -h and lsblk.

Procedure (Command Line – VBoxManage)

```
VBoxManage createvm --name "Ubuntu-VM" --ostype Ubuntu_64 --register
VBoxManage modifyvm "Ubuntu-VM" --cpus 3 --memory 10240 --vram 16 --nic1 nat
VBoxManage createmedium disk --filename "Ubuntu-VM.vdi" --size 10240 --format VDI
VBoxManage storagectl "Ubuntu-VM" --name "SATA" --add sata --controller IntelAhci
VBoxManage storageattach "Ubuntu-VM" --storagectl "SATA" --port 0 --device 0 --type hdd --medium
"Ubuntu-VM.vdi"
VBoxManage storageattach "Ubuntu-VM" --storagectl "SATA" --port 1 --device 0 --type dvddrive --
medium ubuntu-22.04-live-server-amd64.iso
VBoxManage startvm "Ubuntu-VM"
```

Output

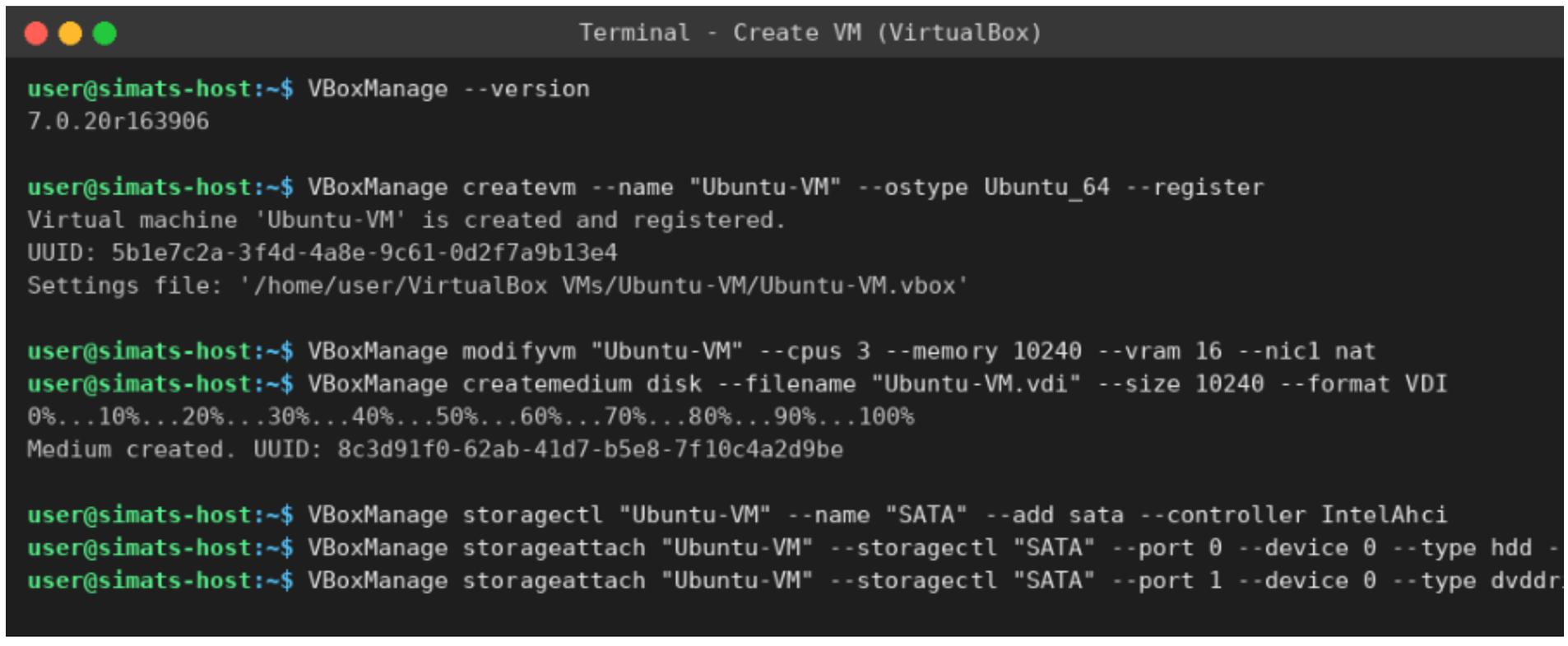


Figure 3.1 – Creating and configuring the VM using VBoxManage

Cl

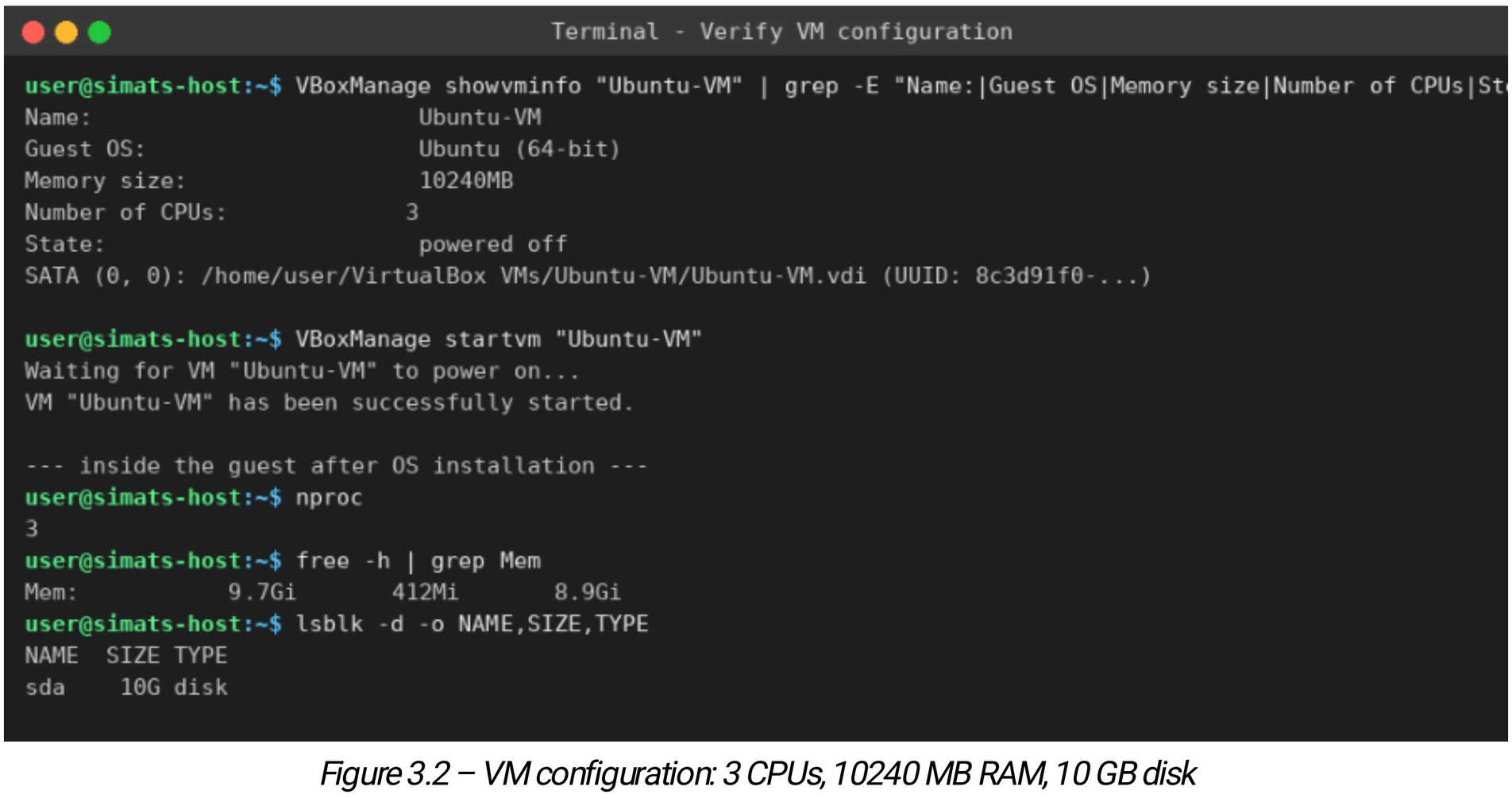


Figure 3.2 – VM configuration: 3 CPUs, 10240 MB RAM, 10 GB disk

Result

A Type-2 hypervisor (VirtualBox) was installed and a virtual machine with 3 CPUs, 10 GB RAM and a 10 GB disk was created, started and verified.

Experiment 4: Virtualization – Snapshot and Clone of a VM

Aim

To create a snapshot of a virtual machine, modify the VM, and test the snapshot by restoring the previous version; also to create a clone of the VM and verify it.

Requirements

- Host OS (Windows/Linux) with Oracle VirtualBox installed
- The Ubuntu-VM created in Experiment 3 (installed and working)

Theory

A snapshot preserves the state of a VM (disk contents, settings and optionally memory) at a given moment, so that the VM can later be rolled back to that point. VirtualBox stores changes made after a snapshot in a differencing disk. A clone is an independent copy of a VM; a full clone copies all virtual disks, while a linked clone shares the parent's disk and stores only differences. Snapshots are useful before software upgrades or risky experiments; clones are useful for replicating test environments.

Procedure (GUI)

1. Start Ubuntu-VM and create a file: echo 'Version 1 - original data' > ~/project.txt.
2. In VirtualBox Manager select the VM, open the Snapshots tab and click Take. Name it Snapshot-v1.
3. Inside the VM modify the data: overwrite project.txt with 'Version 2 - modified' and create extra_file.txt.
4. Power off the VM. In the Snapshots tab right-click Snapshot-v1 and choose Restore (untick 'create a snapshot of the current state' if not required).
5. Start the VM and check that project.txt shows Version 1 and extra_file.txt is gone.
6. Right-click the VM, select Clone, name it Ubuntu-VM-Clone, choose Full clone and Current machine state.
7. Start the cloned VM and confirm that it contains the same data.

Procedure (Command Line)

```
# take snapshot
VBoxManage snapshot "Ubuntu-VM" take "Snapshot-v1" --description "Clean state"
# list snapshots
VBoxManage snapshot "Ubuntu-VM" list
# restore (VM must be powered off)
VBoxManage controlvm "Ubuntu-VM" poweroff
VBoxManage snapshot "Ubuntu-VM" restore "Snapshot-v1"
# clone
VBoxManage clonevm "Ubuntu-VM" --name "Ubuntu-VM-Clone" --register --mode all
VBoxManage startvm "Ubuntu-VM-Clone"
```

Output

```
Terminal - Snapshot and Restore

--- inside the guest (state BEFORE snapshot) ---
user@simats-host:~$ echo 'Version 1 - original data' > ~/project.txt
user@simats-host:~$ cat ~/project.txt
Version 1 - original data

--- on the host ---
user@simats-host:~$ VBoxManage snapshot "Ubuntu-VM" take "Snapshot-v1" --description "Clean state"
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Snapshot taken. UUID: e14a7b90-5c2d-4f38-a6d1-93b8c0f2e741

--- inside the guest (changes AFTER snapshot) ---
user@simats-host:~$ echo 'Version 2 - modified' > ~/project.txt && touch ~/extra_file.txt
user@simats-host:~$ ls ~ && cat ~/project.txt
extra_file.txt  project.txt
Version 2 - modified

--- on the host ---
user@simats-host:~$ VBoxManage controlvm "Ubuntu-VM" poweroff
user@simats-host:~$ VBoxManage snapshot "Ubuntu-VM" list
   Name: Snapshot-v1 (UUID: e14a7b90-5c2d-4f38-a6d1-93b8c0f2e741) *
user@simats-host:~$ VBoxManage snapshot "Ubuntu-VM" restore "Snapshot-v1"
Restoring snapshot 'Snapshot-v1'... 100%
user@simats-host:~$ VBoxManage startvm "Ubuntu-VM"

--- inside the guest after restore ---
user@simats-host:~$ ls ~ && cat ~/project.txt
project.txt
Version 1 - original data
```

Figure 4.1 – Taking a snapshot, modifying the VM and restoring the previous version

```
Terminal - Clone VM and Test

user@simats-host:~$ VBoxManage clonevm "Ubuntu-VM" --name "Ubuntu-VM-Clone" --register --mode all
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Machine has been successfully cloned as "Ubuntu-VM-Clone"

user@simats-host:~$ VBoxManage list vms
"Ubuntu-VM" {5b1e7c2a-3f4d-4a8e-9c61-0d2f7a9b13e4}
"Ubuntu-VM-Clone" {a92c4d17-7e0b-4b65-8f3a-2c1d6e5b90f8}

user@simats-host:~$ VBoxManage startvm "Ubuntu-VM-Clone"
VM "Ubuntu-VM-Clone" has been successfully started.
--- inside the cloned VM ---
user@simats-host:~$ cat ~/project.txt
Version 1 - original data   <-- data preserved in clone
```

Figure 4.2 – Cloning the VM and verifying data in the cloned VM

Result

A snapshot of the VM was created, the VM was modified and then successfully restored to the earlier state. A full clone of the VM was also created and verified.